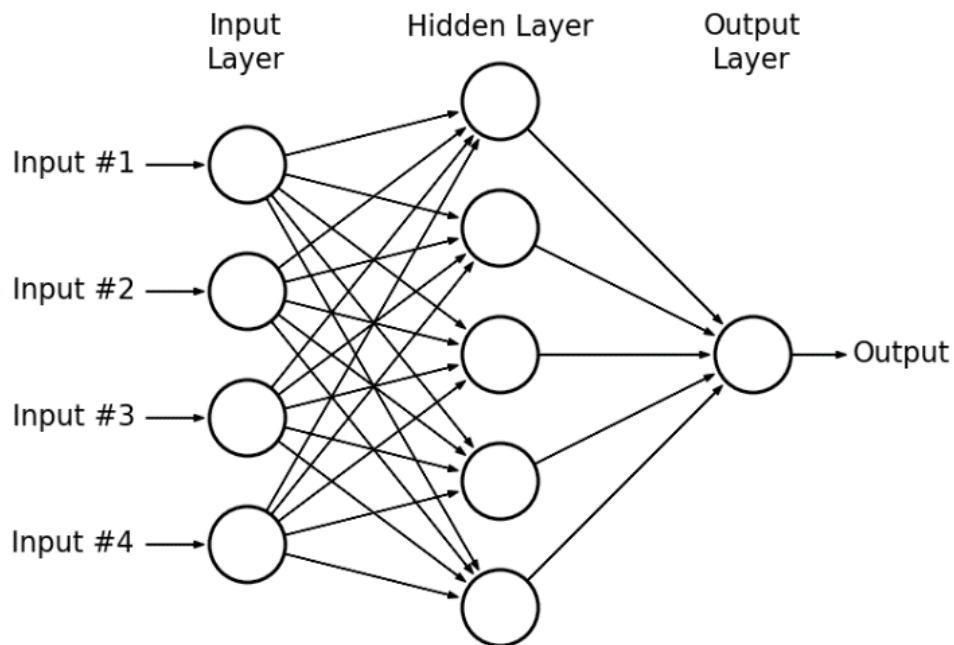


THE PERCEPTRON

A Simple and Complete Guide to the Building Block of Deep Learning



A multilayer perceptron network, made of many small decision units.

Md. Shaon Khan

BSc in Information and Communication Technology
Institute of Information Technology(IIT), Jahangirnagar University

Table of Contents

Contents

Chapter 1: Introduction to the Perceptron	5
1.1 What Is a Perceptron?	5
1.2 From Biological Neuron to Artificial Neuron	5
1.3 The Perceptron as a Decision Maker	5
1.4 How a Perceptron Learns From Its Mistakes	5
1.5 A Simple Limitation to Keep in Mind	6
Chapter 2: How a Perceptron Works	7
Step 1: Take the Inputs	7
Step 2: Multiply Each Input by a Weight	7
Step 3: Add the Bias.....	7
Step 4: Calculate the Weighted Sum (Net Input)	7
Step 5: Send z Through an Activation Function	7
2.1 The Complete Flow	8
Chapter 3: Activation Functions	9
3.1 Why Do We Need an Activation Function?	9
3.2 The Step Function — The Classic Choice.....	9
3.3 Sigmoid Function	10
3.4 Tanh (Hyperbolic Tangent) Function	10
3.5 ReLU (Rectified Linear Unit)	10
3.6 Leaky ReLU	11
3.7 Softmax Function	11
3.8 Comparing All Activation Functions.....	12
3.9 Which Activation Function Is Used Where?	12
Chapter 4: Types of Perceptron	14
4.1 Single-Layer Perceptron (SLP)	14
Key Features of Single-Layer Perceptron	15
4.2 Multi-Layer Perceptron (MLP)	15
Key Features of Multi-Layer Perceptron	16
4.3 Single-Layer Perceptron vs. Multi-Layer Perceptron	16
Chapter 5: How a Perceptron Learns	18
5.1 The Weight Update Formula (Perceptron Learning Rule)	18
5.2 The Bias Update Formula (Perceptron Learning Rule)	18
5.3 The Gradient Descent Formula.....	19

5.4 Perceptron Learning Rule vs. Gradient Descent.....	19
5.5 Can a Single-Layer Perceptron Use Gradient Descent?	20
Chapter 6: Epoch — A Full Training Cycle.....	22
6.1 Why Do We Need Many Epochs?	22
6.2 What Happens Inside One Epoch?	22
6.3 The Role of Epoch in the Perceptron Learning Rule	23
6.4 The Role of Epoch in Gradient Descent	23
6.5 A Simple Example	23
6.6 What Happens If We Use Too Many or Too Few Epochs?	24
Chapter 7: Worked Examples — Training a Perceptron Step by Step	25
7.1 Example: AND Gate.....	25
Epoch 1.....	25
Epoch 2.....	25
Epoch 3 (Checking the Result)	26
7.2 Example: OR Gate	26
Epoch 1.....	27
Epoch 2.....	27
Epoch 3 (Checking the Result)	27
Epoch 4.....	27
7.3 Example: XOR Gate — Where the Perceptron Fails	28
Epoch 1.....	28
Epoch 2.....	28
Why This Happens	29
Why a Multi-Layer Perceptron Can Solve XOR.....	29
Chapter 8: Where the Perceptron Fails.....	31
8.1 Where the Single-Layer Perceptron Fails.....	31
Main Limitations of the Single-Layer Perceptron	31
8.2 Where the Multi-Layer Perceptron Fails.....	32
Main Limitations of the Multi-Layer Perceptron.....	32
8.3 Comparing the Failures of SLP and MLP	33
Chapter 9: Key Insights — The Big Picture.....	34
9.1 Not All Inputs Are Equally Important.....	34
9.2 Bias Gives the Model Flexibility	34
9.3 Learning Happens Gradually, Not Instantly	34
9.4 The Decision Boundary Defines What a Perceptron Can Do	34
9.5 A Single Perceptron Is Weak. Many Perceptrons Together Are Powerful.	35

9.6 A Real-World Example: Loan Approval	35
9.7 Key Insights at a Glance	35
Chapter 10: Building a Perceptron in Python	37
10.1 Function-Based Version	37
10.2 Class-Based Version	38

Chapter 1: Introduction to the Perceptron

1.1 What Is a Perceptron?

A Perceptron is the smallest and simplest unit of an Artificial Neural Network (ANN). It is also the starting point of Deep Learning. A Perceptron is an artificial neuron. Scientists built it by studying how a real neuron works inside the human brain.

In 1958, a scientist named Frank Rosenblatt built the first Perceptron model. His goal was simple. He wanted a machine that could look at some input data, learn from it, and then make a decision or a prediction. Today, even the biggest Deep Learning models are built on this same simple idea. A large model is just many perceptrons working together.

1.2 From Biological Neuron to Artificial Neuron

Inside the human brain, a biological neuron receives signals from other neurons. If the signal is strong enough, the neuron “fires” and sends a new signal forward. A Perceptron copies this idea in a mathematical way. It takes some input numbers, checks how important each one is, adds them together, and then decides whether to “fire” (give an output of 1) or stay quiet (give an output of 0).

1.3 The Perceptron as a Decision Maker

The easiest way to understand a Perceptron is to think of it as a small decision maker. Let us look at one example.

Suppose a university wants to decide if a student should get a scholarship. The university looks at three things about the student:

- GPA (grade average)
- Attendance
- Admission Test Score

These three things become the Inputs of the Perceptron. The Perceptron gives each input a Weight. A higher weight means that input is more important for the final decision. The Perceptron multiplies each input by its weight, adds everything together, and produces one final answer: the student gets the scholarship (output = 1) or does not get it (output = 0).

This is exactly what every Perceptron does. It looks at some data, and it gives one clear decision.

1.4 How a Perceptron Learns From Its Mistakes

A Perceptron does not make perfect decisions from the very first try. When it makes a wrong decision during training, it changes its Weights and Bias a little bit. This process is called the

Perceptron Learning Algorithm. Every time the Perceptron makes a wrong prediction, it updates its weights so that the next prediction can be better. By looking at many training examples again and again, the Perceptron slowly learns to make good decisions. Chapter 5 explains this learning process in full detail.

1.5 A Simple Limitation to Keep in Mind

A single Perceptron can only solve simple problems where the data can be split by drawing one straight line (or, in higher dimensions, one flat surface called a hyperplane). This is called Linearly Separable Data. But many real problems, such as the famous XOR Problem, cannot be solved this way. Chapters 4, 7, and 8 explain this limitation in detail, and how a Multi-Layer Perceptron was built to solve it.

Chapter Summary

A Perceptron is the most basic building block of Artificial Neural Networks and Deep Learning. It works like a small decision maker: it takes inputs, gives each input a weight based on importance, adds them together, and produces one final output. It learns by updating its weights and bias whenever it makes a mistake. A single Perceptron can only solve simple, linearly separable problems, which is why bigger networks were later built on top of this same idea.

Chapter 2: How a Perceptron Works

A Perceptron works in five simple steps. Once you understand these five steps, you understand the core idea behind almost every neural network.

Step 1: Take the Inputs

A Perceptron receives one or more inputs. These inputs are usually written as x_1 , x_2 , up to x_n . Each input is simply a number that represents one piece of information, such as GPA, attendance, or a pixel value in an image.

Step 2: Multiply Each Input by a Weight

Every input has its own Weight, written as w_1 , w_2 , up to w_n . A weight tells the Perceptron how important that input is. A large weight means the input has a strong effect on the final decision. A small weight (close to zero) means the input barely matters.

Step 3: Add the Bias

After multiplying every input by its weight, the Perceptron adds one more number called the Bias (b). The bias is not connected to any input. Its job is to shift the decision boundary left or right (or up or down), which makes the model more flexible. Without a bias, the decision boundary would always be forced to pass through the origin point, which would make it much harder to solve many real problems.

Step 4: Calculate the Weighted Sum (Net Input)

The Perceptron now combines everything into one number, called the Net Input or the Weighted Sum, written as z . The formula is:

$$z = (x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n) + b$$

In short form, this is often written as $z = \sum(x_i \cdot w_i) + b$, where the Greek letter Σ (sigma) simply means “add up all of these terms.”

Step 5: Send z Through an Activation Function

The value of z is not used as the final answer right away. Instead, it is passed into an Activation Function. This function decides whether the neuron should “activate” (send a strong signal forward) and what the final output should be. The classic Perceptron uses a Step Function: if z is greater than a chosen threshold, the output is 1; otherwise, the output is 0 (sometimes -1). This means a classic Perceptron always chooses between two classes, which is called Binary Classification. Chapter 3 covers activation functions in full detail.

2.1 The Complete Flow

Step	What Happens
1	Receive inputs $x_1, x_2, \dots, x_n$
2	Multiply each input by its weight $w_1, w_2, \dots, w_n$
3	Add the bias b
4	Sum everything to get the weighted sum z
5	Pass z through an activation function to get the final output

Chapter Summary

A Perceptron works through five simple steps: it takes inputs, multiplies each input by a weight, adds a bias, calculates the weighted sum z , and finally passes z through an activation function to produce the output. The weights show how important each input is, and the bias makes the model more flexible by shifting its decision boundary.

Chapter 3: Activation Functions

An Activation Function is a mathematical function that takes the weighted sum z as its input and decides the final output of a neuron. In simple words, an activation function decides if a neuron should be “active” (send a strong signal forward) or “inactive” (send little or nothing forward).

The full flow of a neuron looks like this:

Input → Multiply by Weight → Add Bias → Weighted Sum (z) → Activation Function → Final Output

This last step can also be written as $a = f(z)$, where z is the weighted sum, f is the activation function, and a is the final output of the neuron.

3.1 Why Do We Need an Activation Function?

If a network did not use any activation function, every layer would only perform a simple, straight-line (linear) calculation. Even if the network had many hidden layers, the whole network would behave like one single, simple linear equation. It would never be able to learn complex or curved patterns.

An activation function solves this problem. It adds Non-Linearity, which gives the network the power to learn complicated patterns. This is one of the most important ideas in Deep Learning. In short, activation functions are used to:

- Add non-linearity to the model
- Allow the network to learn complex patterns
- Decide which neurons should activate
- Help deep neural networks work correctly

3.2 The Step Function — The Classic Choice

The first and simplest activation function, used in the original Perceptron, is the Step Function. It is also called the Binary Step Function or the Threshold Function. If the weighted sum z is above a chosen threshold, the output is 1. Otherwise, the output is 0.

$$f(z) = 1, \text{ if } z > 0 \quad f(z) = 0, \text{ if } z \leq 0$$

z (weighted sum)	Output
-3	0
-1	0
0	0

z (weighted sum)	Output
2	1
5	1

The Step Function is simple and fast. But it has one big weakness: it is not Differentiable, which means we cannot calculate its slope (derivative) at every point. Because of this, methods like Gradient Descent and Backpropagation cannot be used with it directly. This is why modern Deep Learning almost never uses the plain Step Function anymore, except inside the classic, original Perceptron model.

3.3 Sigmoid Function

The Sigmoid function is smooth and differentiable. It squeezes any input into a value between 0 and 1, which makes it useful for representing probability.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Output range: 0 to 1

Sigmoid is commonly used for binary classification problems and for turning a raw score into a probability. Its main weakness is that in very deep networks, it can cause the Vanishing Gradient Problem, where the learning signal becomes too small to update the weights properly.

3.4 Tanh (Hyperbolic Tangent) Function

The Tanh function looks similar to Sigmoid, but its output range is between -1 and +1 instead of 0 and 1.

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Output range: -1 to 1

Because its output is centered around zero, Tanh often works better than Sigmoid inside hidden layers. However, it can still suffer from the same vanishing gradient problem in very deep networks.

3.5 ReLU (Rectified Linear Unit)

ReLU is the most popular activation function used in modern Deep Learning.

$$f(z) = \begin{cases} 0, & \text{if } z < 0 \\ z, & \text{if } z \geq 0 \end{cases}$$

ReLU simply outputs 0 for every negative input, and it outputs the same value for every positive input. This simple rule makes training much faster and greatly reduces the vanishing gradient problem. This is why ReLU is the default choice for hidden layers in CNNs and most modern deep networks.

3.6 Leaky ReLU

One weakness of ReLU is that it always outputs exactly 0 for negative inputs. Over time, some neurons can get stuck and stop learning completely. This is called the Dead Neuron Problem. Leaky ReLU fixes this by allowing a very small, non-zero output for negative values instead of a hard zero.

$$f(z) = \begin{cases} 0.01z, & \text{if } z < 0 \\ z, & \text{if } z \geq 0 \end{cases}$$

3.7 Softmax Function

Softmax is used in the output layer when a model must choose between more than two classes (Multi-Class Classification).

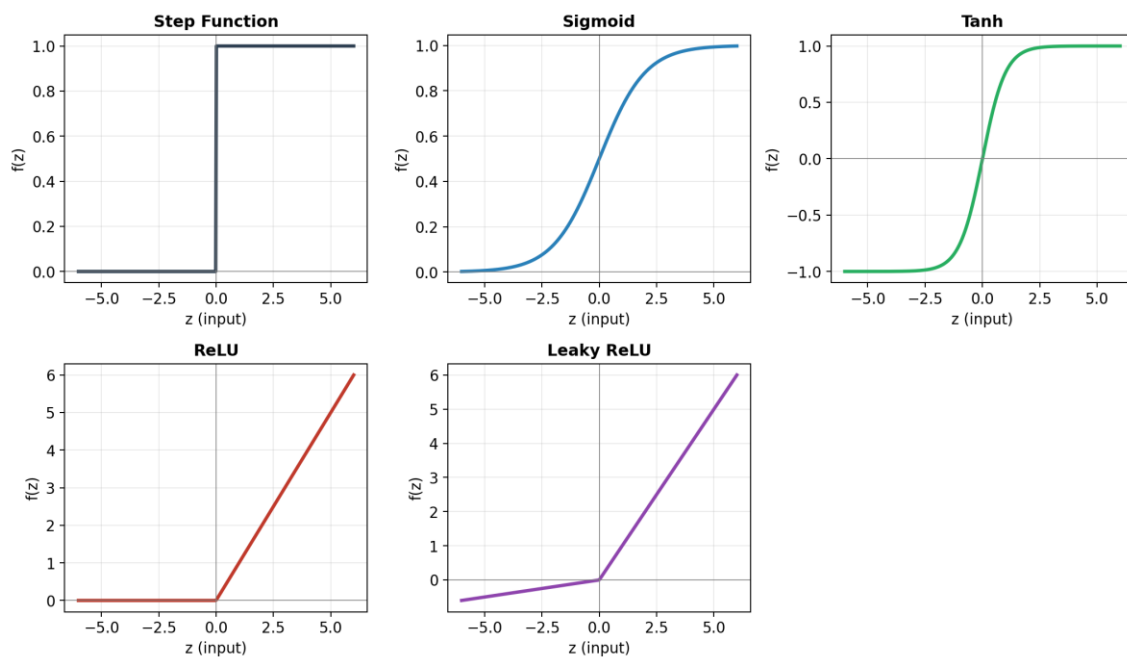
$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Softmax turns a set of raw scores into probabilities that all add up to exactly 1. The class with the highest probability is chosen as the final answer.

Class	Probability
Cat	0.10
Dog	0.80
Bird	0.10

In this example, the model would choose “Dog” because it has the highest probability.

3.8 Comparing All Activation Functions



Shapes of the Step, Sigmoid, Tanh, ReLU, and Leaky ReLU activation functions.

Function	Output Range	Differentiable?	Main Use
Step Function	0, 1	No	Classic Perceptron
Sigmoid	0 to 1	Yes	Binary classification
Tanh	-1 to 1	Yes	Hidden layers (older networks)
ReLU	0 to ∞	Yes	Hidden layers (most popular today)
Leaky ReLU	$-\infty$ to ∞	Yes	Improved version of ReLU
Softmax	0 to 1 (probability)	Yes	Multi-class output layer

3.9 Which Activation Function Is Used Where?

- Single-Layer Perceptron: Step Function
- Binary Classification output layer: Sigmoid
- Multi-Class Classification output layer: Softmax
- Hidden layers (CNNs, MLPs, most deep networks): ReLU
- Alternatives to ReLU: Leaky ReLU, GELU, ELU (for special cases)
- Older neural networks: Tanh

Chapter Summary

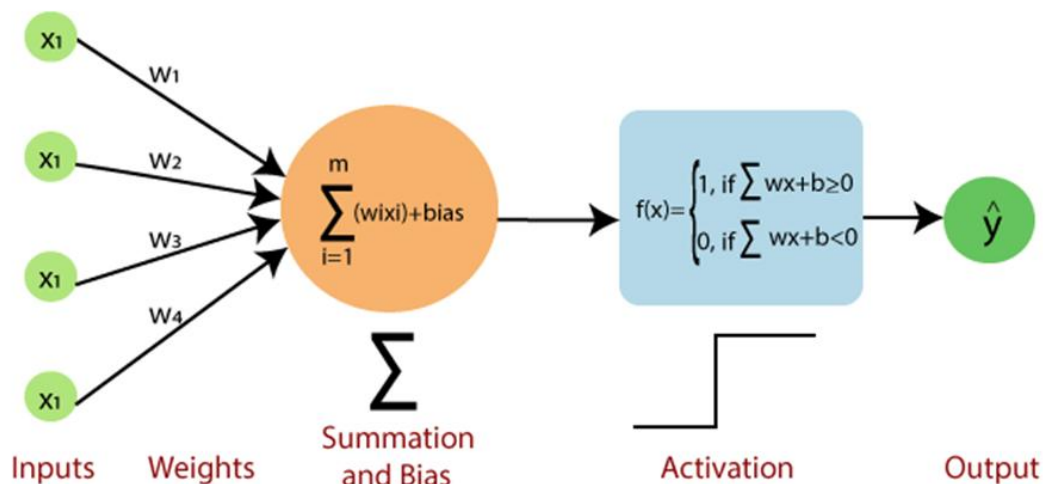
An activation function decides the final output of a neuron and gives a neural network the power to learn non-linear, complex patterns. The classic Perceptron uses the simple Step Function, but it is not differentiable, so it cannot be used with Gradient Descent. Modern Deep Learning mainly uses Sigmoid and Softmax in output layers, and ReLU (or Leaky ReLU) in hidden layers, because these functions are smooth, differentiable, and train much faster.

Chapter 4: Types of Perceptron

Based on structure and ability, Perceptrons are grouped into two main types:

- Single-Layer Perceptron (SLP)
- Multi-Layer Perceptron (MLP)

Both types belong to the family of Artificial Neural Networks. But they are very different in structure, learning power, and real-world use.



A biological neuron compared with a Single-Layer Perceptron and a Multi-Layer Perceptron.

4.1 Single-Layer Perceptron (SLP)

A Single-Layer Perceptron is the simplest and most basic model of an Artificial Neural Network. It has only an Input Layer and an Output Layer. There is no Hidden Layer in between. This means the input data goes directly to the output layer without passing through any middle processing layer.

Each input has its own weight. All the weighted inputs are added together with a bias to form the net input, and then a Step Function usually decides the final output.

A Single-Layer Perceptron is built for Binary Classification. It can choose between two classes, such as Yes/No, True/False, or Spam/Not Spam. It works very well when the data is Linearly Separable, which means the two classes can be split apart using one straight line (or one flat hyperplane in higher dimensions).

During training, whenever the Perceptron makes a wrong prediction, the Perceptron Learning Algorithm updates its weight and bias. Over time, this update helps the model find the correct decision boundary.

The biggest weakness of the Single-Layer Perceptron is that it cannot solve Non-Linearly Separable problems. The most famous example is the XOR Problem, where a single straight line can never correctly separate the two classes. Chapters 7 and 8 explain this weakness with full worked examples.

Key Features of Single-Layer Perceptron

- Has only one output layer
- Has no hidden layer
- Best suited for binary classification
- Can only solve linearly separable data
- Simple structure, so training is fast
- Cannot solve difficult problems such as XOR

4.2 Multi-Layer Perceptron (MLP)

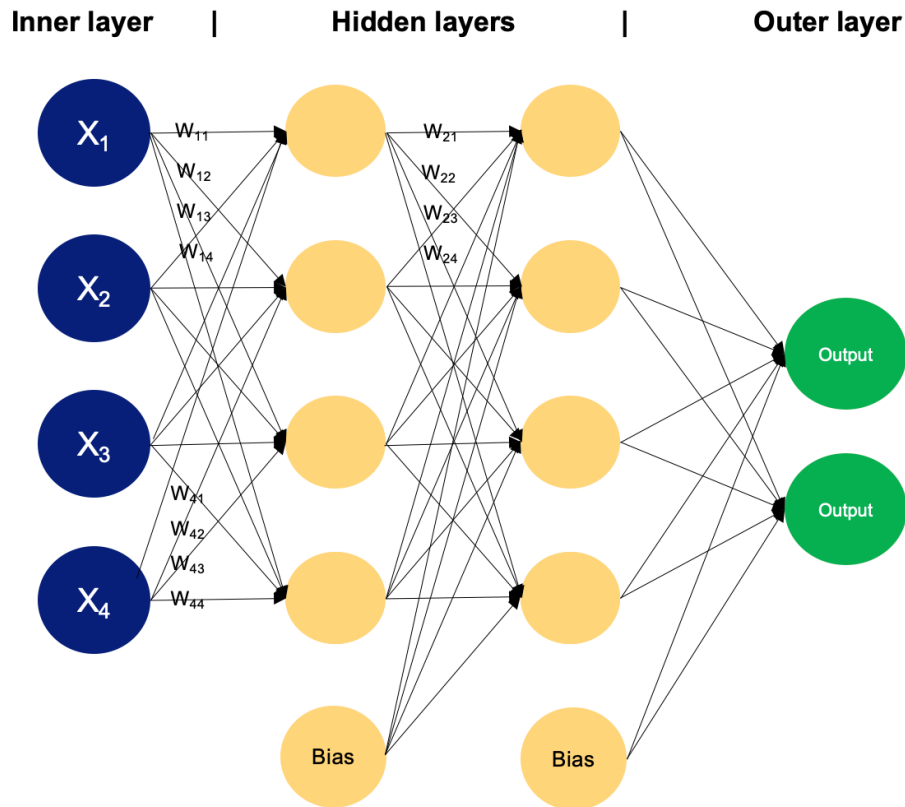
The Multi-Layer Perceptron was built to remove the limitations of the Single-Layer Perceptron. It is one of the most fundamental models in modern Deep Learning. An MLP has an Input Layer, one or more Hidden Layers, and an Output Layer. Adding a hidden layer gives the model the power to learn much more complex patterns in the data.

Every neuron in an MLP takes input from the layer before it, multiplies each input by a weight, adds a bias to get the weighted sum, and then passes this sum through a Non-Linear Activation Function such as ReLU, Sigmoid, or Tanh. This process repeats through every hidden layer until the output layer produces the final prediction.

Because of its hidden layers, a Multi-Layer Perceptron can successfully classify both Linearly Separable and Non-Linearly Separable data. It can solve difficult problems such as the XOR Problem, because a hidden layer can learn the complex relationships hidden inside the data.

Training an MLP needs more than the simple Perceptron Learning Rule. It uses the Backpropagation Algorithm together with Gradient Descent Optimization. First, the model makes a prediction. Then it calculates the error of that prediction. Then it spreads (“backpropagates”) that error backward through every layer, updating the weights and bias along the way. This whole process repeats many times, and with every repetition (called an Epoch), the model's accuracy slowly improves.

Today, the idea behind the Multi-Layer Perceptron is used in almost every modern Deep Learning application, including Computer Vision, Image Classification, Speech Recognition, Natural Language Processing (NLP), Medical Diagnosis, Recommendation Systems, and Generative AI.



A deep Multi-Layer Perceptron, with several hidden layers stacked between the input and output layers.

Key Features of Multi-Layer Perceptron

- Has one or more hidden layers
- Can learn complex, curved patterns
- Can handle both linearly and non-linearly separable data
- Trained using the Backpropagation Algorithm
- Forms the foundation of Deep Learning
- Can reach high accuracy, but training takes more time

4.3 Single-Layer Perceptron vs. Multi-Layer Perceptron

Feature	Single-Layer Perceptron (SLP)	Multi-Layer Perceptron (MLP)
Layers	Input + Output	Input + Hidden + Output
Hidden Layer	None	One or more
Learning Power	Simple patterns only	Complex patterns
Data Type	Linearly separable only	Linear and non-linear
XOR Problem	Cannot solve	Can solve
Training Method	Perceptron Learning Rule	Backpropagation + Gradient Descent

Feature	Single-Layer Perceptron (SLP)	Multi-Layer Perceptron (MLP)
Common Activation	Step Function	ReLU, Sigmoid, Tanh, Softmax
Accuracy	Relatively lower	Relatively higher
Typical Use	Simple binary classification	Image, speech, NLP, deep learning

Chapter Summary

Perceptrons are mainly divided into two types. A Single-Layer Perceptron is the simplest model, with no hidden layer, and it can only classify linearly separable data. A Multi-Layer Perceptron adds one or more hidden layers, which lets it learn complex features and solve non-linearly separable problems such as XOR. Nearly every modern deep neural network, including CNNs, RNNs, and Transformers, is an advanced extension of the Multi-Layer Perceptron idea.

Chapter 5: How a Perceptron Learns

The main goal of training a Perceptron is to find the right Weight (w) and Bias (b) values so that the model can classify data correctly. When a Perceptron makes a wrong prediction during training, it changes its weight and bias. There are two main methods used for this change. The first is the Perceptron Learning Rule, used mainly in a Single-Layer Perceptron. The second is Gradient Descent, used in Multi-Layer Perceptrons and modern Deep Learning models. Both methods try to reduce the model's error, but they work in different ways, as this chapter explains.

5.1 The Weight Update Formula (Perceptron Learning Rule)

A weight is a number that shows how important an input feature is. If an input has a strong effect on the prediction, it gets a larger weight. When training produces a wrong prediction, the weight is changed so that future predictions can be more accurate.

The Perceptron weight update formula is:

$$w_{\text{new}} = w_{\text{old}} + \eta (y - \hat{y}) \cdot x$$

Symbol	Meaning
w_{old}	the current weight
w_{new}	the updated weight
η (eta)	the learning rate
y	the actual (correct) output
$\hat{y}$ (y-hat)	the predicted output
x	the input feature

Here, $(y - \hat{y})$ is called the Prediction Error. If a prediction is correct, the error is 0 and the weight does not change. If a prediction is wrong, the error is not 0, so the weight is adjusted. In short, the weight update step changes the importance of each input. If a feature is causing wrong decisions, its weight is reduced or increased so that, over many rounds of training, the model's accuracy slowly improves.

5.2 The Bias Update Formula (Perceptron Learning Rule)

Bias is an extra parameter that shifts the decision boundary left, right, up, or down. Without a bias, the decision boundary would always be forced to pass through the origin, which would make it impossible to solve many problems correctly.

The bias update formula is:

$$b_{\text{new}} = b_{\text{old}} + \eta (y - \hat{y})$$

Notice that this formula does not include the input x . This is because bias is not tied to any single feature; instead, it shifts the entire decision boundary. Its job is to make the model more flexible. Sometimes changing the weights alone is not enough to find the correct decision boundary; changing the bias moves the whole boundary so the model can classify the data correctly.

5.3 The Gradient Descent Formula

The Perceptron Learning Rule only works well for a Single-Layer Perceptron. But a Multi-Layer Perceptron, and modern Deep Learning models in general, have a huge number of weights and biases. Updating all of these parameters together requires a more general and more powerful method, called the Gradient Descent Optimization Algorithm.

The goal of Gradient Descent is to reduce the value of a Loss Function step by step, until it reaches its lowest possible value (the minimum).

$$w = w - \alpha \frac{\partial L}{\partial w}$$

$$b = b - \alpha \frac{\partial L}{\partial b}$$

Symbol	Meaning
w, b	the weight and the bias
α (alpha)	the learning rate
L	the loss function
$\partial L / \partial w, \partial L / \partial b$	the gradient of the loss with respect to w and b

The gradient shows which direction, and by how much, we should change a value in order to reduce the loss. Gradient Descent always updates the weight and bias in the opposite direction of the gradient, because moving opposite to the gradient is what actually reduces the loss.

At every training step, Gradient Descent calculates the loss and uses its gradient to update the weight and bias. This process repeats again and again, over many epochs, so the model gradually learns to make better predictions and the loss keeps getting smaller.

5.4 Perceptron Learning Rule vs. Gradient Descent

Feature	Perceptron Learning Rule	Gradient Descent
Main Goal	Fix weight & bias after a wrong prediction	Minimize the loss function

Feature	Perceptron Learning Rule	Gradient Descent
Used In	Single-Layer Perceptron	Multi-Layer Perceptron & Deep Learning
Uses Error	Directly, as $(y - \hat{y})$	Through the gradient of a loss function
Update Based On	Prediction error	Calculus (partial derivatives)
Needs a Loss Function?	No	Yes
Needs Backpropagation?	No	Yes, in MLPs
Solves Complex Problems?	No	Yes, more easily
Works for Non-linear Models?	No	Yes
Typical Use	Simple binary classification	CNN, RNN, Transformer, MLP, Deep Learning

In short: the Perceptron Learning Rule learns by directly looking at the Prediction Error. Gradient Descent learns by looking at the Gradient of the Loss Function. Gradient Descent is a more general and more powerful optimization method, and it is used everywhere, from single layers to the deepest modern neural networks.

5.5 Can a Single-Layer Perceptron Use Gradient Descent?

Yes, it can — but the answer needs a small explanation. Historically, and in most textbooks, a Single-Layer Perceptron is trained using the Perceptron Learning Rule described above. But mathematically, Gradient Descent can also be used, as long as the model uses a Differentiable Loss Function.

The problem is that the classic Perceptron uses a Step Activation Function, and the Step Function is not differentiable, meaning we cannot calculate its slope. Because of this, Gradient Descent cannot be applied directly to the classic Perceptron.

However, if a single-layer model replaces the Step Function with a differentiable activation function such as Sigmoid or Tanh, and pairs it with a suitable loss function (such as Mean Squared Error or Cross-Entropy Loss), then Gradient Descent can be used to update its weight and bias, exactly like the formulas shown in Section 5.3.

So, in practice: if we mean the Classical Single-Layer Perceptron with a Step Function, the standard method is the Perceptron Learning Rule. But if we mean a Single-Layer Neural Network, such as a Logistic Regression model, which uses a differentiable activation and a loss function, then Gradient Descent is the standard method.

Model Type	Standard Training Method
Classical Single-Layer Perceptron (Step Function)	Perceptron Learning Rule
Single-Layer Neural Network (e.g., Logistic Regression)	Gradient Descent

Chapter Summary

A Perceptron learns by adjusting its weight and bias whenever it makes a wrong prediction. The Perceptron Learning Rule updates weight and bias directly using the prediction error, and it is used for the classic Single-Layer Perceptron. Gradient Descent instead reduces a loss function step by step using calculus, and it is used for Multi-Layer Perceptrons and almost all modern Deep Learning models. A Single-Layer Perceptron can only use Gradient Descent if it swaps its Step Function for a differentiable activation function such as Sigmoid.

Chapter 6: Epoch — A Full Training Cycle

An Epoch is one of the most important ideas in Machine Learning and Deep Learning. An epoch means that every single sample in the Training Dataset has been passed through the model exactly one time. In other words, when a model looks at the entire training dataset once, from the first sample to the last, and updates its weight and bias, that one full pass is called one Epoch.

Here is a simple way to picture it. Reading a book once, from the first page to the last page, is one “reading cycle.” In the same way, a model looking at the whole training dataset once is one Epoch. During an epoch, the model learns from the data, makes predictions, calculates errors, and updates its weight and bias. But usually, one single epoch is not enough for the model to learn well. That is why training is repeated on the same dataset again and again, so the model can slowly improve its predictions.

6.1 Why Do We Need Many Epochs?

When training starts, the model's weight and bias are usually set to random or simple starting values. Because of this, the model's early predictions are usually wrong, and the error is high. After each epoch, the model updates its weight and bias, so predictions in the next epoch become a little better than before. As many epochs pass, the error gradually goes down, and the accuracy gradually goes up.

In short, the purpose of every new epoch is to learn from the mistakes of the epoch before it, and make a better decision next time.

6.2 What Happens Inside One Epoch?

Inside each epoch, the following steps normally happen:

- The whole training dataset enters the model.
- The model makes a prediction for every sample.
- The error between the prediction and the actual output is calculated.
- That error is used to update the weight and bias.
- Once the dataset is finished, one epoch is complete.
- The next epoch begins again, using the newly updated weight and bias.

This cycle continues either for a fixed number of epochs, or until the model reaches a good enough result.

6.3 The Role of Epoch in the Perceptron Learning Rule

In a Single-Layer Perceptron, every sample in the training dataset is used once during each epoch. If a sample is classified incorrectly, the weight and bias are updated right away, using the Perceptron Learning Rule:

$w_{new} = w_{old} + \eta(y - \hat{y})x$ $b_{new} = b_{old} + \eta(y - \hat{y})$
--

Once one epoch is finished, the model starts training again on the same dataset, but this time with its newly updated weight and bias. This continues over many epochs, until the model finds the correct decision boundary, or until the maximum number of epochs is reached.

6.4 The Role of Epoch in Gradient Descent

Gradient Descent also uses the entire training dataset in every epoch. But instead of using the error directly, it first calculates a Loss Function, then uses the gradient of that loss to update the weight and bias:

$w = w - \alpha \frac{\partial L}{\partial w}$
$b = b - \alpha \frac{\partial L}{\partial b}$

With each new epoch, the loss usually becomes smaller than before, and the model moves closer and closer to the best possible weight and bias values. In Deep Learning, training can run for hundreds or even thousands of epochs.

6.5 A Simple Example

Suppose a training dataset has 1,000 samples, and the model is trained for 10 epochs.

- Epoch 1: the model sees all 1,000 samples for the first time. The error is high.
- Epoch 2: using the new weight and bias, the model sees the same 1,000 samples again. The error drops a little.
- Epoch 3: the model sees the full dataset again. Predictions keep improving.
- ... and so on ...
- Epoch 10: after learning many times, the model reaches a much better accuracy.

In total, the model has processed $10 \times 1,000 = 10,000$ samples. This is a bit like studying for an exam: the same material is reviewed again and again, the night before the test. The only difference is that a model never complains about it.

6.6 What Happens If We Use Too Many or Too Few Epochs?

If the number of epochs is too small, the model does not get enough chances to learn. This is called Underfitting. On the other hand, if the number of epochs is too large, the model may start memorizing the training data instead of truly learning from it. Then it performs poorly on new, unseen data. This is called Overfitting.

Because of this, choosing the right number of epochs is important: enough to let the model learn well, but not so many that it starts memorizing. In practice, engineers use tools such as Validation Loss, Validation Accuracy, and Early Stopping to decide the right number of epochs to use.

Chapter Summary

An Epoch is one complete pass of the model through the entire training dataset. Both the Perceptron Learning Rule and Gradient Descent update the weight and bias after every epoch, which slowly reduces error and increases accuracy. The difference is that the Perceptron Learning Rule updates weight and bias directly from the prediction error, while Gradient Descent updates them using the gradient of a loss function. In both methods, running the training over many epochs is what allows the model to learn step by step and make better predictions.

Chapter 7: Worked Examples — Training a Perceptron Step by Step

This chapter trains a Single-Layer Perceptron on three classic logic gates: AND, OR, and XOR. In every example, we use the same setup: Learning Rate $\eta = 1$, and starting values $w_1 = 0$, $w_2 = 0$, $b = 0$. We use the same two formulas from Chapters 2 and 5:

$$z = x_1w_1 + x_2w_2 + b$$

$$\hat{y} = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

$$w_i^{new} = w_i^{old} + \eta(y - \hat{y})x_i$$

$$b^{new} = b^{old} + \eta(y - \hat{y})$$

In every table below, each row shows one training sample. The last three columns show the weight and bias values after that sample has been processed, which are then carried forward into the next row.

7.1 Example: AND Gate

The AND gate outputs 1 only when both inputs are 1.

x_1	x_2	Actual Output (y)
0	0	0
0	1	0
1	0	0
1	1	1

Epoch 1

x_1	x_2	y	z	$\hat{y}$	Error	New w_1	New w_2	New b
0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0
1	1	1	0	0	1	1	1	1

End of Epoch 1: $w_1 = 1$, $w_2 = 1$, $b = 1$.

Epoch 2

x_1	x_2	y	z	$\hat{y}$	Error	New w_1	New w_2	New b
0	0	0	1	1	-1	1	1	0
0	1	0	1	1	-1	1	0	-1
1	0	0	0	0	0	1	0	-1
1	1	1	0	0	1	2	1	0

End of Epoch 2: $w_1 = 2$, $w_2 = 1$, $b = 0$.

Epoch 3 (Checking the Result)

Input (x_1, x_2)	z	Predicted	Actual	Correct?
(0,0)	0	0	0	Yes
(0,1)	1	1	0	No
(1,0)	2	1	0	No
(1,1)	3	1	1	Yes

Two samples are still wrong, so training continues. After a few more epochs of the same update process, the model settles on the values $w_1 = 2$, $w_2 = 1$, $b = -2$. Let us check this final result against the whole dataset:

x_1	x_2	$z = x_1w_1 + x_2w_2 + b$	Predicted	Actual
0	0	-2	0	0
0	1	-1	0	0
1	0	0	0	0
1	1	1	1	1

Every prediction is now correct. Training is complete, with a final accuracy of 100%. The final parameters are $w_1 = 2$, $w_2 = 1$, $b = -2$.

7.2 Example: OR Gate

The OR gate outputs 1 when at least one input is 1.

x_1	x_2	Actual Output (y)
0	0	0
0	1	1
1	0	1
1	1	1

Epoch 1

x_1	x_2	y	z	$\hat{y}$	Error	New w_1	New w_2	New b
0	0	0	0	0	0	0	0	0
0	1	1	0	0	1	0	1	1
1	0	1	1	1	0	0	1	1
1	1	1	2	1	0	0	1	1

End of Epoch 1: $w_1 = 0$, $w_2 = 1$, $b = 1$.

Epoch 2

x_1	x_2	y	z	$\hat{y}$	Error	New w_1	New w_2	New b
0	0	0	1	1	-1	0	1	0
0	1	1	1	1	0	0	1	0
1	0	1	0	0	1	1	1	1
1	1	1	3	1	0	1	1	1

End of Epoch 2: $w_1 = 1$, $w_2 = 1$, $b = 1$.

Epoch 3 (Checking the Result)

Input (x_1, x_2)	z	Predicted	Actual	Correct?
(0,0)	1	1	0	No
(0,1)	2	1	1	Yes
(1,0)	2	1	1	Yes
(1,1)	3	1	1	Yes

One sample, (0,0), is still wrong, so one more update is needed.

Epoch 4

Processing sample (0,0) with current parameters $w_1 = 1$, $w_2 = 1$, $b = 1$ gives $z = 1$, predicted = 1, actual = 0, error = -1. This updates the parameters to $w_1 = 1$, $w_2 = 1$, $b = 0$. Checking the whole dataset again with these values:

x_1	x_2	z	Predicted	Actual
0	0	0	0	0
0	1	1	1	1
1	0	1	1	1

x_1	x_2	z	Predicted	Actual
1	1	2	1	1

Every prediction is now correct. Training is complete, with a final accuracy of 100%. The final parameters are $w_1 = 1$, $w_2 = 1$, $b = 0$.

7.3 Example: XOR Gate — Where the Perceptron Fails

The XOR gate outputs 1 only when the two inputs are different from each other. This example shows exactly why a Single-Layer Perceptron cannot solve the XOR Problem.

x_1	x_2	Actual Output (y)
0	0	0
0	1	1
1	0	1
1	1	0

Epoch 1

x_1	x_2	y	z	$\hat{y}$	Error	New w_1	New w_2	New b
0	0	0	0	0	0	0	0	0
0	1	1	0	0	1	0	1	1
1	0	1	1	1	0	0	1	1
1	1	0	2	1	-1	-1	0	0

End of Epoch 1: $w_1 = -1$, $w_2 = 0$, $b = 0$.

Epoch 2

x_1	x_2	y	z	$\hat{y}$	Error	New w_1	New w_2	New b
0	0	0	0	0	0	-1	0	0
0	1	1	0	0	1	-1	1	1
1	0	1	0	0	1	0	1	2
1	1	0	3	1	-1	-1	0	1

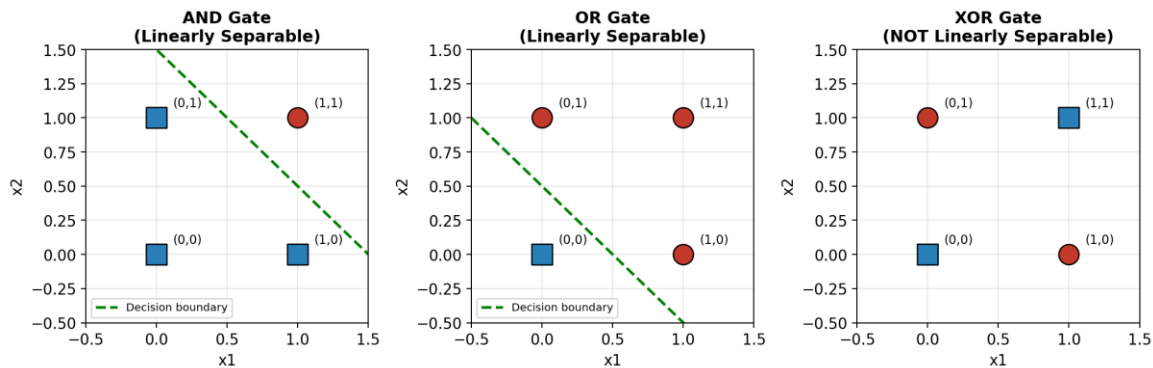
End of Epoch 2: $w_1 = -1$, $w_2 = 0$, $b = 1$.

If we keep training for many more epochs, a repeating pattern appears. Once the parameters are fixed so that (0,1) becomes correct, (1,1) becomes wrong. Once (1,1) is fixed, (1,0) becomes wrong. Once (1,0) is fixed, (0,1) becomes wrong again. The weight and bias keep changing round

after round, but no single combination of w_1 , w_2 , and b can ever classify all four samples correctly at the same time.

Why This Happens

The reason becomes clear when we plot the XOR data on a graph. Look at the position of each class:



AND and OR gates can be separated by one straight line. XOR cannot — its two classes are mixed diagonally, so no single straight line can separate them.

For the AND gate and the OR gate, one straight line can cleanly separate the outputs of 0 from the outputs of 1. But for the XOR gate, the two output-1 points, (0,1) and (1,0), sit on opposite corners from each other, while the two output-0 points, (0,0) and (1,1), sit on the other two opposite corners. No single straight line can put both (0,1) and (1,0) on one side while keeping (0,0) and (1,1) on the other side. This is what it means for XOR to be a Non-Linearly Separable Dataset.

Because of this, no matter how many epochs we run, whether 10, 100, or even 1,000, a Single-Layer Perceptron will never converge on the XOR problem. In other words, the Perceptron does not converge for XOR data.

Why a Multi-Layer Perceptron Can Solve XOR

A Multi-Layer Perceptron has one or more Hidden Layers. A hidden layer transforms the original input into a new feature space, where the same XOR data becomes linearly separable. Once the data is transformed this way, the output layer can easily draw a simple boundary and classify it correctly. This is exactly why an MLP can succeed at the XOR problem, while a Single-Layer Perceptron cannot.

Chapter Summary

A Single-Layer Perceptron was successfully trained on the AND gate and the OR gate, because both are linearly separable problems; in both cases, the model reached 100% accuracy after a few epochs. But when trained on the XOR gate, the Perceptron never converges, no matter how many epochs are used, because XOR is a non-linearly separable problem. This exact limitation is what led to the creation of the Multi-Layer Perceptron, which uses hidden layers to transform

the data into a space where it becomes linearly separable, and can therefore solve the XOR problem successfully.

Chapter 8: Where the Perceptron Fails

Even though the Perceptron is the foundation of modern Deep Learning, it cannot solve every kind of problem. Its limitations depend on its structure (architecture), its learning method, and the nature of the data it is given. The Single-Layer Perceptron and the Multi-Layer Perceptron fail for different reasons. The Single-Layer Perceptron has fairly basic limitations, while the Multi-Layer Perceptron is far more powerful, but it still runs into its own set of challenges on certain complex problems.

8.1 Where the Single-Layer Perceptron Fails

The biggest limitation of a Single-Layer Perceptron (SLP) is that it can only correctly classify Linearly Separable Data. If the two classes of data cannot be separated by drawing one straight line (in two dimensions) or one hyperplane (in higher dimensions), then a Single-Layer Perceptron simply cannot learn the correct decision boundary. Even after running training for many epochs, the model will never converge, and it will fail to reach a good accuracy.

As shown in detail in Chapter 7, the most well-known example of this failure is the XOR Problem. Because the two classes in the XOR dataset cannot be separated by a single straight line, the Perceptron keeps updating its weight and bias forever, without ever finding a solution that works for every sample. This limitation was a very important event in the history of Artificial Intelligence, and it became the main reason the Multi-Layer Perceptron was later invented.

Beyond this, a Single-Layer Perceptron is also naturally limited to Binary Classification. It cannot handle Multi-Class Classification (choosing between more than two categories) on its own. It also cannot learn complex feature relationships, curved decision boundaries, or non-linear patterns. Because of these limits, it is not useful for real, modern AI problems such as image recognition, speech recognition, or Natural Language Processing (NLP).

Main Limitations of the Single-Layer Perceptron

- Can only handle linearly separable data
- Cannot solve the XOR Problem
- Cannot learn non-linear decision boundaries
- Does not directly support multi-class classification
- Cannot understand complex feature relationships
- Cannot solve complex problems such as image, speech, or NLP tasks

8.2 Where the Multi-Layer Perceptron Fails

The Multi-Layer Perceptron (MLP) removed many of the limitations of the Single-Layer Perceptron. By using hidden layers together with non-linear activation functions, it can successfully learn non-linearly separable data as well. But this does not mean the MLP can solve every problem. Many real-world, complex problems still create challenges for an MLP.

First, training an MLP requires much more computation and much more time. As the number of layers and neurons increases, the number of weights and biases increases too. This means training needs more memory, more processing power, and more time to complete.

Second, if the training dataset is small but the model itself is very large, the MLP can suffer from Overfitting. This happens when the model memorizes the training data extremely well, but performs poorly on new, unseen data.

Third, in very deep neural networks, a problem called Vanishing Gradient or Exploding Gradient can appear. During backpropagation, if the gradient becomes extremely small or extremely large, the weight updates stop working correctly. This makes training slow, or sometimes completely unstable.

Finally, an MLP is generally not the best architecture for image data or sequential data. For example, understanding the spatial structure of an image is usually done far more effectively with a Convolutional Neural Network (CNN). Similarly, understanding sequential information, such as language, text, or time-series data, is usually handled far better by architectures such as a Recurrent Neural Network (RNN), LSTM, GRU, or Transformer. So even though an MLP can sometimes be used for these tasks, it is rarely the best or most efficient solution.

Main Limitations of the Multi-Layer Perceptron

- Training is relatively slow and computationally expensive
- Larger models can suffer from overfitting
- Vanishing and exploding gradient problems can appear
- Choosing the right hyperparameters (learning rate, number of hidden layers, number of neurons) is difficult
- CNNs are usually more effective for image data
- RNNs, LSTMs, or Transformers are usually more effective for sequential data (text, speech, time-series)
- Large datasets and powerful hardware may be required

8.3 Comparing the Failures of SLP and MLP

Issue	Single-Layer Perceptron (SLP)	Multi-Layer Perceptron (MLP)
Non-linear data	Cannot handle	Can handle
XOR Problem	Fails	Succeeds
Complex pattern learning	Fails	Succeeds
Image recognition	Not effective	Possible, but CNN is better
Sequential data	Not effective	Possible, but RNN / LSTM / Transformer is better
Training speed	Fast	Slower
Overfitting risk	Generally low	Can be high
Gradient problems	None (no backpropagation used)	Vanishing / exploding gradient possible
Computational cost	Low	High

Chapter Summary

A Single-Layer Perceptron is only built for linearly separable data, so it fails on the XOR Problem and any other non-linear or multi-class problem. The Multi-Layer Perceptron was created to fix this, using hidden layers and backpropagation to learn complex patterns. However, the MLP is not perfect either — large MLP models can suffer from overfitting, vanishing or exploding gradients, high computational cost, and they are often outperformed by more specialized architectures such as CNNs for images and RNNs, LSTMs, or Transformers for sequential data. This is why modern Deep Learning chooses a different neural network architecture depending on the type of problem being solved.

Chapter 9: Key Insights — The Big Picture

Understanding the deeper idea behind the Perceptron is one of the most important steps in learning Deep Learning. Many people think of a Perceptron as just a mathematical formula or an algorithm. But in reality, it is best understood as an Artificial Decision-Making Unit. Its whole job is to look at different inputs and reach one clear decision. This is very similar to how a biological neuron in the human brain receives different signals and decides whether to pass a signal on to the next neuron. A Perceptron does the same thing mathematically: it takes inputs, uses weight and bias to make a decision, and produces an output of 0 or 1.

9.1 Not All Inputs Are Equally Important

One of the most important ideas behind the Perceptron is that not every input carries the same importance. Each input has its own Weight, which shows how much that input matters for the final decision. During training, if an input strongly contributes to a correct prediction, its weight grows larger. If an input causes wrong decisions, its weight shrinks. Over time, the model automatically learns which information matters most, and which information matters less.

9.2 Bias Gives the Model Flexibility

Another key part of the Perceptron is the Bias. Without a bias, the decision boundary would always be forced to pass through the origin, which would make it impossible to solve many problems correctly. The bias shifts the decision boundary, which makes the model far more flexible and able to work well across many different kinds of datasets.

9.3 Learning Happens Gradually, Not Instantly

A Perceptron never makes perfect decisions from the very beginning. At the start, its weight and bias are usually set to some initial or random values. The model then makes a prediction for every training sample. Whenever the prediction is wrong, the weight and bias are updated based on the error. This update does not finish in one step; it happens epoch after epoch, on the same dataset, again and again. With every new update, the model learns a little more from its previous mistakes, and slowly builds a better decision boundary. This gradual, repeated learning process is the true foundation of Machine Learning.

9.4 The Decision Boundary Defines What a Perceptron Can Do

A Perceptron works by drawing a Decision Boundary that separates data into different classes. If the data can be separated by a straight line or a flat hyperplane, a single Perceptron can classify it successfully. But if the data is Non-Linearly Separable, such as the XOR Problem explained in Chapter 7, then a single Perceptron alone is not enough. This exact limitation is what led to the

creation of the Multi-Layer Perceptron (MLP), which uses hidden layers to learn much more complex patterns.

9.5 A Single Perceptron Is Weak. Many Perceptrons Together Are Powerful.

Perhaps the most important insight of all is this: a single Perceptron has very limited power on its own, but when thousands, or even millions, of perceptrons work together in many connected layers, they can form an extremely powerful neural network. A single Perceptron might only be able to make one simple decision. But when many perceptrons are stacked together across multiple layers, the resulting network can recognize images, understand language, diagnose diseases, and even power modern Generative AI models. This is exactly why the Perceptron is called the Building Block of modern Artificial Intelligence.

9.6 A Real-World Example: Loan Approval

Consider a real-life example. Suppose a bank needs to decide whether to approve a loan for an applicant. The bank looks at inputs such as Income, Credit Score, Employment Status, and Existing Debt. A Perceptron assigns a weight to each of these features, based on how important it is, adds a bias, calculates the weighted sum, and finally decides: Loan Approved (1) or Loan Rejected (0). Through training on many past examples, the model gradually learns which features matter the most when it comes to approving a loan.

9.7 Key Insights at a Glance

- A Perceptron is the most basic building block of an Artificial Neural Network.
- It is an artificial neuron that analyzes inputs and produces a decision.
- The importance of each input is controlled by its weight.
- Bias helps shift the decision boundary and makes the model more flexible.
- When a prediction is wrong, weight and bias are updated, and the model learns.
- Training runs across many epochs on the same dataset to reduce error.
- A Single-Layer Perceptron can only solve linearly separable problems.
- Non-linearly separable problems need a Multi-Layer Perceptron.
- Modern Deep Learning models are built from huge numbers of perceptrons working together.

Chapter Summary

At its core, a Perceptron is a learning-based decision maker. It decides how important each input is using weight, adjusts its decision boundary using bias, and learns from every wrong prediction by updating its weight and bias repeatedly. A single Perceptron can only solve simple, linear problems, but when many perceptrons are connected together across multiple layers, they

become capable of learning extremely complex patterns. This is exactly why the Perceptron is considered the foundation of modern Deep Learning and Artificial Intelligence.

Chapter 10: Building a Perceptron in Python

This final chapter shows how to build a Single-Layer Perceptron from scratch in Python, using no external Machine Learning libraries. Building it yourself is a great way to fully understand every step covered in this book: the weighted sum, the step activation function, the weight update, and the bias update. Two versions are shown below: a simple function-based version, and a cleaner, class-based version.

10.1 Function-Based Version

This version trains a Perceptron on the OR Gate dataset, and prints the result after every training step.

```
# Single-Layer Perceptron From Scratch (OR Gate)

# OR Gate dataset: [x1, x2, actual_output]
dataset = [
    [0, 0, 0],
    [0, 1, 1],
    [1, 0, 1],
    [1, 1, 1]
]

# Initial weight and bias
w1 = 0
w2 = 0
b = 0

# Learning rate
lr = 1

# Number of training epochs
epochs = 10

# Step activation function
def step(z):
    if z > 0:
        return 1
    return 0

# Training loop
for epoch in range(epochs):
    print(f"\n===== Epoch {epoch + 1} =====")
    for x1, x2, y in dataset:
        # Forward pass
        z = x1 * w1 + x2 * w2 + b
        prediction = step(z)

        # Calculate error
        error = y - prediction

        # Update weight and bias
        w1 = w1 + lr * error * x1
        w2 = w2 + lr * error * x2
        b = b + lr * error
```

```

        print(f"Input=({x1},{x2}) | Actual={y} | Prediction={prediction} | Error={error}")
        print(f"Updated -> w1={w1}, w2={w2}, b={b}")

# Final check after training
print("\n===== Final Prediction =====")
for x1, x2, y in dataset:
    z = x1 * w1 + x2 * w2 + b
    prediction = step(z)
    print(f"Input=({x1},{x2}) | Prediction={prediction} | Actual={y}")

print("\nFinal Weights and Bias")
print(f"w1 = {w1}")
print(f"w2 = {w2}")
print(f"b = {b}")

```

10.2 Class-Based Version

The same logic can be organized more cleanly using a Python class. This structure is closer to how real Machine Learning libraries are built.

```

# Single-Layer Perceptron Using a Class (OR Gate)

class Perceptron:
    def __init__(self, learning_rate=1, epochs=10):
        self.w1 = 0
        self.w2 = 0
        self.b = 0
        self.lr = learning_rate
        self.epochs = epochs

    # Step activation function
    def step(self, z):
        if z > 0:
            return 1
        return 0

    # Training function
    def train(self, dataset):
        for epoch in range(self.epochs):
            print(f"\n===== Epoch {epoch + 1} =====")
            for x1, x2, y in dataset:
                # Forward pass
                z = (x1 * self.w1) + (x2 * self.w2) + self.b
                prediction = self.step(z)

                # Error
                error = y - prediction

                # Update weight and bias
                self.w1 = self.w1 + self.lr * error * x1
                self.w2 = self.w2 + self.lr * error * x2
                self.b = self.b + self.lr * error

            print(f"Input=({x1},{x2}) | Actual={y} | Prediction={prediction} | Error={error}")
            print(f"Updated -> w1={self.w1}, w2={self.w2}, b={self.b}")

    # Final prediction check

```

```

def evaluate(self, dataset):
    print("\n===== Final Prediction =====")
    for x1, x2, y in dataset:
        z = (x1 * self.w1) + (x2 * self.w2) + self.b
        prediction = self.step(z)
        print(f"Input=({x1},{x2}) | Prediction={prediction} | Actual={y}")
    print("\nFinal Weights and Bias")
    print(f"w1 = {self.w1}")
    print(f"w2 = {self.w2}")
    print(f"b = {self.b}")

# OR Gate dataset
dataset = [
    [0, 0, 0],
    [0, 1, 1],
    [1, 0, 1],
    [1, 1, 1]
]

# Create and train the model
model = Perceptron(learning_rate=1, epochs=10)
model.train(dataset)
model.evaluate(dataset)

```

Chapter Summary

Both versions of the code implement the exact same logic used throughout this book: calculate the weighted sum, apply the step activation function, measure the error, and update the weight and bias. Building a Perceptron from scratch, without any external library, is one of the best ways to fully understand how a neural network actually learns.

End of Book